

DSC 102 Final Exam Cheat Sheet

Emphasis on Lectures 9-14, with Foundations & FRQ Models

Page 1: Foundations (Lectures 1-8 Condensed)

Hardware & Memory Hierarchy

Levels (Fastest → Slowest): Registers → CPU Caches → DRAM → SSD → HDD. **Key Device Properties:**

- **DRAM:** Volatile, byte-addressable, fast. No random vs. seq. dichotomy.
- **HDD:** Persistent, block-addressable. HUGE random vs. sequential dichotomy (due to seek/rotation).
- **SSD:** Persistent, block-addressable. Faster than HDD, less of a dichotomy but sequential is still better. Degrades with heavy writes.

Locality of Reference: Exploited by caches. Temporal (time), Spatial (space). **Pareto Frontier:** Set of non-dominated models. E.g., for Cost vs. Error (lower is better), a model is dominated if another is better on BOTH axes. M3(15% err, 20K cost) is dominated by M1(12% err, 10K cost).

Operating System (OS)

Process vs. Thread: Process = running program, private address space. Thread = execution unit within a process, SHARES address space. **Virtual Memory & Paging:**

- **Page:** Fixed-size chunk of virtual memory.
- **Page Frame:** Slot in physical DRAM to hold a page.
- **Page Fault:** CPU requests page not in DRAM. OS loads it from disk.

	Policy	Preempt?	Key Trait
CPU Scheduling Policies:	FIFO	No	Simple. Suffers from Convoy Effect .
	SJF	No	Optimal avg. TT if lengths known.
	SCTF	Yes	Preemptive SJF. Optimal avg. TT.
	RR	Yes	Fair. Uses time quantum.

Metrics: TT = Completion - Arrival. RT = Start - Arrival.

Parallelism & Cloud

Shared-X Architectures:

- **Shared Memory:** All CPUs share RAM (your laptop).
- **Shared Disk:** Nodes have private RAM, share storage (Oracle RAC).
- **Shared Nothing:** Nodes share nothing. (Hadoop/Spark).

Amdahl's Law: Max speedup is limited by the serial part. **Speedup vs. Scaleup:** Speedup (Strong): Fixed problem, more resources. Scaleup (Weak): Problem grows with resources. **Cloud Models:** IaaS (EC2), PaaS (RDS), SaaS (SageMaker), FaaS (Lambda).

Data Representation & Models

Bits/Bytes: n bits → 2^n values. k items → $\lceil \log_2 k \rceil$ bits. **Floating Point:** Prone to precision errors. Lower precision (float16) saves space/memory bandwidth, crucial for DL. **Data Models:**

- **Structured:** Relation (unordered), DataFrame (ordered rows), Matrix.
- **Semi-structured:** JSON, XML. **Unstructured:** Text, images.

File Formats for Structured Data:

- **CSV/TSV:** Text, row-store. Inefficient.
- **Parquet/ORC:** Binary, **columnar**, compressed. For analytics.
- **Arrow:** An **in-memory** columnar format standard. Zero-copy reads.

Page 2: Scalable Data Access & Algorithms (Lectures 9-10)

Regimes of Scalable Data Access

1. **Single-Node Disk:** Data > DRAM, on one machine. Access pages from local disk.
2. **Remote Read:** Access pages from network storage (e.g., S3) to a single machine.
3. **Distributed Memory:** Data fits in total DRAM of a cluster.
4. **Distributed Disk:** Data > total cluster DRAM.

The Problem: Data > DRAM

Paging: OS breaks file on disk into **pages**; loads them into **frames** in DRAM. **Caching:** Keeping frequently used pages in DRAM. **Spilling:** If a "dirty" page (modified) is evicted, it must be written to disk (write I/O). **Cache Replacement Policy:** Algorithm (e.g., LRU) to decide which page to evict. **I/O Cost:** Number of pages read/written. Goal is to **minimize I/O cost**.

Critical Concept: Data Layouts on Disk

The way 2D data is serialized to a file dramatically impacts I/O

Layout	How it's Stored	Best For... (Lowest I/O)
Row-Store (CSV)	(A1,B1,C1), (A2,B2,C2) ... Pages contain full rows.	Queries needing many columns from few rows. Ex: <code>SELECT * WHERE A=5</code>
Column-Store (Parquet)	(A1,A2...), (B1,B2...), (C1,C2...) Pages contain one column.	Analytical queries needing few columns from many rows. Ex: <code>SELECT AVG(C)</code> .
Tiled/Hybrid (Modin)	Data split into rectangular blocks.	Mix of workloads, matrix algebra.

I/O Cost Example: `SELECT SUM(B) FROM R`: Table R has 3 columns (A, B, C) and takes up 6 pages total on disk.

- **Row-Store Cost:** Must read all pages. I/O cost = **6 pages**.
- **Column-Store Cost:** Each column takes 2 pages. Only read pages for column B. I/O cost = **2 pages**.

Frameworks for Single-Node Scaling

Dask DataFrame: Scales Pandas via row-store partitions. **Modin DataFrame:** Scales Pandas via tiled storage layout.

Scaling Relational Operations

Simple Aggregates (MAX(C)) & Selections (WHERE B=val):

- **Algorithm:** A simple **filescan**. Read file page-by-page, apply logic.
- **I/O Cost:** 1 full pass over relevant data. (Column-store is much better for aggregates).

GROUP BY Aggregates (GROUP BY A, SUM(D)):

- **Algorithm:** Filescan with an in-memory **hash table**.
- **The "Big" Problem:** Fails if the hash table (number of unique groups) is too large for DRAM.
- **I/O Cost:** 1 filescan (if hash table fits in RAM).

Scaling ML Algorithms

Batch	vs.	Stochastic Gradient Descent (BGD)	vs.
	Aspect	BGD	SGD
SGD):	Gradient	Sum of gradients over ENTIRE dataset.	Gradient from one mini-batch .
	Update	1 model update per epoch.	Many updates per epoch.
	I/O	1 full filescan per epoch.	1 full filescan per epoch.
	Cost	Not needed.	Crucial. Data must be shuffled before each epoch.
	Shuffle		
	Pros	Stable convergence.	Faster convergence, escapes local minima, workhorse of DL.
	Cons	Very slow, high I/O.	Noisy updates.

SGD Iterations Calculation: Total Iterations = (#Epochs) * (Total Examples / Batch Size) Ex: 100M examples, batch size 50, 20 epochs → $20 \times (100M/50) = 40M$ updates.

Page 3: Data Parallelism & Dataflow Systems (Lectures 11-13)

Data Parallelism

Partitioning a large dataset across multiple machines (workers) and having them compute on their local partition simultaneously. **Shared-Nothing Architecture:** Dominant paradigm. Each worker node has its own private CPU, RAM, and Disk. **Sharding:** Physically partitioning the data across nodes.

- **Round-Robin:** Simple, good load balance.
- **Hash Partitioning:** $\text{worker} = \text{hash}(\text{key}) \% k$. Crucial for parallel joins.
- **Range Partitioning:** Good for range queries.

Bulk Synchronous Parallelism (BSP): A common protocol for coordinating data-parallel execution.

1. Manager gives a task to all workers.
2. Workers compute in parallel on their local data.
3. **Barrier:** All workers must finish and report back.
4. Manager aggregates results and starts the next stage.

Straggler Problem: In BSP, the stage is only as fast as its slowest worker.

Distributed File Systems (DFS)

Hadoop Distributed File System (HDFS):

- **NameNode:** The "manager". Stores all metadata. **Single point of failure.**
- **DataNodes:** The "workers". Store actual data blocks.
- **Data Blocks:** Data is split into large blocks (default '128MB').
- **Replication:** Each block is replicated (default '3x') on different DataNodes. This provides **fault tolerance**.

MapReduce & Hadoop

Programming Model: User provides two functions:

- **map(key, value):** Runs independently on each record. Emits (new_key, new_value) pairs.
- **reduce(key, list_of_values):** Aggregates all values for a given key.

Execution Flow: Input → Mappers → **Shuffle & Sort** (by key) → Reducers → Output. **Key Weakness:** MapReduce writes all intermediate data from mappers to HDFS. This makes it **extremely slow** for iterative algorithms or multi-step workflows. **Fault Tolerance:** Re-executing failed tasks on replicated data.

Apache Spark

Key Idea vs. Hadoop: Spark keeps intermediate data **in-memory**, avoiding the slow disk I/O of MapReduce. **Lazy Evaluation:** Spark builds a Directed Acyclic Graph (DAG) of operations but does not execute anything until an **action** is called.

- **Transformations:** map, filter, groupBy. Lazy, build the DAG.
- **Actions:** count, collect, save. Trigger the computation.

Lineage-Based Fault Tolerance: Spark's key innovation. Instead of replicating intermediate data, Spark remembers the sequence of transformations (the **lineage**). If a data partition is lost, Spark can simply **recompute** it from the original data using its lineage.

Spark APIs: RDD vs. DataFrame

Aspect	RDD API	DataFrame / SQL API
Abstraction	Low-level. Distributed collection of opaque objects. No schema.	High-level. Distributed table with named columns and a schema.
Data Type	Best for unstructured data.	Best for structured data.
Optimization	None. You are responsible.	Automatic optimization via the Catalyst Optimizer (projection/predicate pushdown, join reordering).
Verdict	Powerful but hard. Rarely used.	Almost always preferred. Easier, faster, and more optimized.

Data Lake

A loose coupling of data file formats (like Parquet on S3) and query processing engines (like Spark, Presto). Contrasts with tightly-coupled RDBMSs.

Page 4: Model Building Systems (Lecture 14)

The ML Model Building Lifecycle

A high-level view of the process: **Data Prep** → **Feature Engineering** → **Algorithm/Arch. Selection** → **Hyperparameter Tuning** → **Deployment**.

Scaling K-Means Clustering

Lloyd's algorithm is iterative. Each iteration has two steps:

1. **Assignment Step:** For each point, find the nearest centroid. This is a "map-only" style job. Each point can be processed independently.
2. **Update Step:** For each cluster, compute the mean of all points assigned to it. This is a classic **GROUP BY** operation.

This iterative process is a great fit for dataflow systems like Spark, which can cache the data between iterations to speed it up.

Scaling Feature Engineering (FE) Tasks

The challenge is how to perform these transformations on data larger than RAM. The key is to identify if the task needs global information or can be done one tuple at a time.

1. **Tuple-level / 1-Pass / "Map-only" Tasks:** These are the easiest to scale. They can be applied to each row/record independently during a single file scan. No communication between workers needed.

- **Feature Interactions (Polynomials):** Creating $x_1 \times x_2$ only needs the values from a single row.
- **Temporal Feature Extraction:** Extracting `month` from a `date` column.
- **Applying Pre-trained Embeddings:** Looking up an embedding vector for a word in a row.

2. **Global Stats / 2-Pass Tasks:** These tasks require information about the entire dataset before the transformation can be applied. This usually requires a 2-pass algorithm in a data-parallel system.

- **Pass 1 (MapReduce):** Scan the entire dataset to compute global statistics (e.g., `mean`, `stddev`, `min`, `max`, list of all unique categories).
- **Pass 2 (Map-only):** Scan the entire dataset again, using the computed stats to transform each row.

Examples requiring 2 passes:

- **Standardization ($(x - \text{mean}) / \text{std}$):** Needs global `mean` and `stddev`.
- **Min-Max Scaling:** Needs global `min` and `max`.
- **One-Hot Encoding:** Needs the global vocabulary of all possible categories.
- **Binning/Discretization (by quantiles):** Needs a global histogram/sorted list to determine bin boundaries.
- **Bag-of-Words (Text):** Needs the global vocabulary of all words.

3. **Complex / Iterative Tasks:** These tasks are algorithms in their own right and require specialized scalable implementations.

- **Dimensionality Reduction (PCA/SVD):** These are iterative matrix factorization algorithms.
- **Feature Selection (Wrapper/Embedded):** These involve training many models, which is itself a complex task. Can often be parallelized as a task-parallel problem.

Learned Features & "Software 2.0"

Deep Learning (DL): The big win of DL is that it performs **automatic feature extraction**. The neural network learns the relevant features from raw data (e.g., pixels, text) as part of its training process, replacing much of the manual FE work.

- **CNNs** learn features from images.
- **RNNs/Transformers** learn features from text.
- **GNNs** learn features from graphs.

Software 2.0: A buzzword for this paradigm shift, where instead of hand-crafting complex logic (manual FE), we specify a model architecture and let it learn the logic from data.

Page 5: Free Response Question Molds & Examples

These templates cover the most common quantitative problems on the final exam.

Mold 1: Amdahl's Law & Speedup Calculation

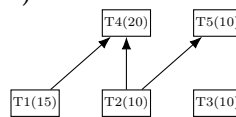
Example: A program takes 90 mins on 1 core. 30% is inherently sequential. What is the runtime on 9 cores and the overall speedup?

1. **Identify Inputs:** $T_1 = 90$ min, $N = 9$, Sequential fraction $(1 - P) = 0.3 \implies P = 0.7$.
2. **Calculate Time for Serial Part:** This part does not speed up. $T_{\text{serial}} = T_1 \times (1 - P) = 90 \times 0.3 = 27$ min.
3. **Calculate Time for Parallel Part:** This part gets linear speedup. $T_{\text{parallel}} = (T_1 \times P)/N = (90 \times 0.7)/9 = 63/9 = 7$ min.
4. **Calculate Total New Time (T_N):** $T_N = T_{\text{serial}} + T_{\text{parallel}} = 27 + 7 = \mathbf{34}$ min.
5. **Calculate Speedup (S_N):** $S_N = T_1/T_N = 90/34 \approx \mathbf{2.65x}$.

Mold 2: Task Graph Analysis

Example: Given a task graph with 3 workers. Tasks: T1(15), T2(10), T3(10), T4(20), T5(10). Dependencies: T1→T4, T2→T4, T2→T5. What is the lowest completion time?

1. **Draw/Visualize Graph & Find Critical Path (CP):**



Paths to sinks: T1→T4 (35), T2→T4 (30), T2→T5 (20), T3 (10). CP length is **35 tu**.

2. **Calculate Total Work (T_1):** $T_1 = 15 + 10 + 10 + 20 + 10 = 65$ tu.
3. **Schedule on N=3 Workers (Manual Simulation):**
 - **t=0:** T1, T2, T3 are ready. Assign W1:T1, W2:T2, W3:T3.
 - **t=10:** W2, W3 finish. T5 needs T2 (done). T4 needs T1 (not done) & T2 (done). Only T5 is ready. Assign T5 to W2.
 - **t=15:** W1 finishes. T4 is now ready. Assign T4 to W1.
 - **Schedule:** W1: T1(0-15) → T4(15-35). W2: T2(0-10) → T5(10-20). W3: T3(0-10).
 - **Final Check:** Last task finishes at time 35. $T_3 = \mathbf{35}$ tu.
4. **Calculate Speedup:** $S_3 = T_1/T_3 = 65/35 \approx 1.86x$.

Mold 3: I/O Cost Calculation (Single Node)

Example: A 120 GB dataset (500M rows, 3 float64 columns: Y, X1, X2) is in a row-store file. No caching. 1. What is the I/O cost to compute quadratic features and write to a new file? 2. What is the I/O cost to train a BGD model for 3 epochs?

1. **Part 1: Quadratic Features**
 - **Read Pass:** Tuple-level op. Row-store requires reading entire file. Read I/O = **120 GB**.
 - **Write Pass:** New file has 6 cols vs 3, so it's 2x larger. Write I/O = $120 \times 2 = \mathbf{240}$ GB.
 - **Total I/O = 120 GB (read) + 240 GB (write) = 360 GB.**
2. **Part 2: BGD Training for 3 Epochs**
 - BGD requires one full filescan per epoch.
 - **Total Read I/O = (# Epochs) × (File Size) = $3 \times 120 = \mathbf{360}$ GB.**

Mold 4: MapReduce Algebraic Aggregate

Example: Compute average of $A=[1,5,5,9,15,20,30]$ on 3 workers. Shards: W1:[5,20], W2:[15,1], W3:[5,9,30]. Manager only sums partial results. What should each worker return?

1. **Goal & Constraint:** Global average. Manager can only SUM.
2. **Algebra:** Global Avg = $(\sum A_i)/N$. We need partial results P_j such that $\sum P_j = (\sum A_i)/N$. A correct formulation is for each worker to return its local sum, weighted by the total population size: $P_j = (\text{Local Sum}_j)/N_{\text{total}}$.
3. **Calculations:** $N_{\text{total}} = 7$.
 - **Worker 1:** Local Sum = $5 + 20 = 25$. $P_1 = 25/7 \approx \mathbf{3.5714}$.
 - **Worker 2:** Local Sum = $15 + 1 = 16$. $P_2 = 16/7 \approx \mathbf{2.2857}$.
 - **Worker 3:** Local Sum = $5 + 9 + 30 = 44$. $P_3 = 44/7 \approx \mathbf{6.2857}$.
4. **Verification:** $\sum P_j = 12.1428$. Global Avg = $85/7 \approx 12.1428$. It works.

Page 6: Comprehensive Question Dump (Samples + New)

True/False Questions

- An SQL query is a string. **(T)**
- If no worker has idle time in task parallelism, we get linear speedup. **(T)**
- Serverless can reduce resource wastage vs classical IaaS. **(T)**
- In scaleup (weak scaling), dataset size is fixed. **(F)** (It grows)
- All threads of a multi-threaded process share the same address space. **(T)**
- Float16 arithmetic is associative. **(F)**
- A compiler converts assembly to human-readable code. **(F)** (Reverse)
- Prefetching from DRAM to cache exploits locality. **(T)**
- Vendor lock-in is a key disadvantage of public clouds. **(T)**
- Tensor arithmetic is not amenable to SIMD. **(F)** (It's a perfect fit)
- Dask scales well to arbitrarily large task graphs. **(F)** (Overhead)
- CPU caches have a random vs sequential access latency dichotomy. **(F)** (HDD trait)

MCQs: Data Access, Layouts, Scaling

- Which is NOT one of the 4 main regimes of scalable data access? (A) Single-node disk (B) Distributed memory (C) **CPU cache-only** (D) Remote read
- Which statement about SGD is TRUE? (A) It uses the entire dataset for each model update. (B) **It relies on shuffling data for effective training.** (C) It has a more stable convergence path than BGD.
- You want to compute **AVG(price)** on a very large table. Which data layout provides the lowest disk I/O cost? (A) Row-store (B) **Column-store** (C) JSON
- Which FE task typically requires at least two full passes over a large dataset? (A) Computing polynomial features. (B) **Standardizing a numeric column.** (C) Applying a pre-trained text embedding.

MCQs: Parallelism & Dataflow Systems

- What is the primary advantage of Spark over MapReduce? (A) Better fault-tolerance. (B) **It performs intermediate computations in-memory.** (C) More restrictive programming model.
- Why is the Spark DataFrame API preferred over RDDs? (A) It supports unstructured data better. (B) **It enables automatic query optimization.** (C) It's the only API with lazy evaluation.
- How does HDFS primarily achieve fault tolerance? (A) Error-correcting codes. (B) **By replicating data blocks across multiple DataNodes.** (C) Through lineage.
- The architecture used by Spark/Hadoop with independent nodes is: (A) Shared Memory (B) Shared Disk (C) **Shared Nothing**
- In a task graph, the minimum completion time for a workload on any number of workers is determined by: (A) The total number of tasks. (B) Degree of parallelism. (D) **The length of the critical path.**
- Spinning out an EC2 cluster and reading data from S3 to each node's DRAM is what form of parallelism? (A) Shared Nothing (B) **Shared Disk** (C) Shared Memory.
- Which tool is custom-designed for tensor dataflow graphs? (A) RDBMSs (B) Pandas (C) **PyTorch.**

MCQs: OS, Hardware, Cloud

- Which is typically considered SaaS? (A) EC2 (B) S3 (C) EBS (D) Lambda (E) **SageMaker.**
- Which device has a random vs sequential access latency dichotomy? (A) CPU Caches (B) DRAM (C) **Magnetic Hard Disk** (D) Flash SSD.
- Which component of disk access latency is primarily affected by RPM? (A) **Rotation delay** (B) Seek time (C) Transfer time.
- What is the OS term for a virtual slot in DRAM holding data from disk? (A) **Page frame** (B) File frame (C) Register.
- Which scheduling policy is preemptive and designed primarily for fairness? (A) FIFO (B) SJF (C) SCTF (D) **Round Robin**

MCQs: Data Models & General Knowledge

- Which is NOT one of the "3 Vs of Big Data"? (A) Variety (B) **Vitality** (C) Velocity (D) Volume.
- Which is NOT a typical format for structured data? (A) CSV (B) JSON (C) TSV (D) **JPEG.**
- Which 2D structured data model has no in-built notion of row ordering? (A) Matrix (B) **Relation** (C) DataFrame.
- How many positive integers have exactly 2 decimal digits and the same 2 hex digits? (A) **0** (B) 9 (C) 10. (Proof: $10x + y = 16x + y \implies x = 0$, but 2-digit number means $x \neq 0$).
- Models: M1(12%,10K), M2(8%,40K), M3(15%,20K), M4(5%,90K). Which is NOT pareto-optimal? (A) M1 (B) M2 (C) **M3** (D) M4. (M1 dominates M3).